MERN + Python + AI/ML — Complete Interview Cheatsheet

Muhammad Faizan Yousaf | WAMO Labs Round 2 | April 26, 2026

PART 1 — MERN STACK

Letter	Technology	Role	Port (default)
M	MongoDB	NoSQL Database — stores JSON-like documents	27017
E	Express.js	Backend web framework for Node.js — handles routing & middleware	—
R	React	Frontend UI library — builds component-based interfaces	3000
N	Node.js	JavaScript runtime — runs JS on the server (not browser)	—

M — MongoDB

Q: What is MongoDB? How is it different from MySQL?

➜ MongoDB stores data as JSON-like documents (BSON) in collections — no fixed schema. MySQL stores data in tables with fixed columns and rows. MongoDB is flexible and horizontally scalable. MySQL is relational with ACID transactions.

```
// MongoDB document
{ _id: ObjectId(), name: "Faizan", skills: ["JS", "Python"], age: 22 }
// MySQL equivalent — rigid, normalized
users table: id | name | age
skills table: user_id | skill
```

Q: MongoDB CRUD operations

➜ Create, Read, Update, Delete — the four basic operations every developer must know.

```
// Create
db.users.insertOne({ name: "Faizan", age: 22 })
// Read
db.users.find({ age: { $gt: 20 } }) // find where age > 20
db.users.findOne({ name: "Faizan" })
// Update
db.users.updateOne({ name: "Faizan" }, { $set: { age: 23 } })
// Delete
db.users.deleteOne({ name: "Faizan" })
```

Q: What is Mongoose? Why use it with MongoDB?

➜ Mongoose is an ODM (Object Data Modeling) library for MongoDB in Node.js. It adds schema validation, type checking, and helper methods on top of raw MongoDB driver.

```
const userSchema = new mongoose.Schema({
name: { type: String, required: true },
email: { type: String, unique: true },
age: { type: Number, min: 0 }
});
const User = mongoose.model('User', userSchema);
// Now you get validation, .save(), .find(), etc.
const user = new User({ name: 'Faizan', email: 'f@g.com', age: 22 });
await user.save();
```

Q: SQL vs NoSQL — when to choose which?

➜ SQL: structured data, complex relationships, strong consistency needed (banking, ERP). NoSQL: flexible schema, high write volume, hierarchical data, horizontal scaling (social media, real-time apps, catalogs).

E — Express.js

Q: What is Express.js and why use it?

➜ Express is a minimal Node.js web framework. It handles routing, middleware, request/response cycle. Without it, you'd write everything from scratch in raw Node http module.

```
const express = require('express');
const app = express();
app.use(express.json()); // parse JSON body
app.get('/users', (req, res) => {
res.json({ users: [] });
});
app.listen(5000, () => console.log('Server running'));
```

Q: What is middleware in Express?

➜ Functions that run between receiving a request and sending a response. They have access to req, res, and next(). Used for auth, logging, parsing, error handling.

```
// Custom middleware
app.use((req, res, next) => {
console.log(req.method, req.url); // log every request
next(); // pass to next middleware or route
});
// Auth middleware
function authMiddleware(req, res, next) {
const token = req.headers.authorization;
if (!token) return res.status(401).json({ error: 'Unauthorized' });
next();
}
app.get('/protected', authMiddleware, (req, res) => { ... });
```

Q: REST API methods — what each does

➜ GET: read data. POST: create new resource. PUT: replace entire resource. PATCH: update part of resource. DELETE: remove resource.

```
app.get('/users', getAllUsers); // read all
app.get('/users/:id', getUserById); // read one
app.post('/users', createUser); // create
app.put('/users/:id', replaceUser); // replace all fields
app.patch('/users/:id', updateUser); // update some fields
app.delete('/users/:id', deleteUser); // delete
```

Q: HTTP Status Codes — must know

➜ 200 OK. 201 Created. 400 Bad Request. 401 Unauthorized. 403 Forbidden. 404 Not Found. 500 Internal Server Error.

```
res.status(200).json({ data }); // success, return data
res.status(201).json({ created }); // resource created
res.status(400).json({ error }); // client sent bad data
res.status(401).json({ error }); // not logged in
res.status(403).json({ error }); // logged in but no permission
res.status(404).json({ error }); // resource not found
res.status(500).json({ error }); // server crashed
```

Q: What is JWT authentication? How does it work?

➜ User logs in → server validates credentials → server creates a signed JWT token → sends to client. Client stores token, sends it in every request header. Server verifies signature — no DB lookup needed per request.

```
// Login — issue token
const token = jwt.sign({ userId: user._id }, process.env.SECRET, { expiresIn: '7d' });
res.json({ token });
// Protected route — verify token
const token = req.headers.authorization?.split(' ')[1]; // 'Bearer '
const decoded = jwt.verify(token, process.env.SECRET);
req.user = decoded; // attach user to request
```

■ *JWT = header.payload.signature — all base64 encoded. Never store sensitive data in payload.*

R — React

Q: What is React? What problem does it solve?

➜ React is a UI library that breaks interfaces into reusable components. It uses a virtual DOM to update only what changed — instead of re-rendering the entire page. Solves: complex UI state management and performance.

Q: What is the Virtual DOM?

➜ React keeps a lightweight copy of the real DOM in memory. On state change, it diffs old vs new virtual DOM (reconciliation) and updates ONLY the changed nodes in the real DOM. Much faster than updating everything.

Q: useState and useEffect — most important hooks

➜ useState: stores local component state — changing it triggers re-render. useEffect: runs side effects (API calls, subscriptions) after render. Dependency array controls when it re-runs.

```
const [count, setCount] = useState(0);
const [data, setData] = useState(null);
useEffect(() => {
fetch('/api/data')
.then(r => r.json())
.then(setData);
}, []); // [] = run once on mount
useEffect(() => {
console.log('count changed:', count);
}, [count]); // run every time count changes
```

Q: Props vs State — difference?

➜ Props: data passed FROM parent TO child — read-only inside child, child cannot change them. State: data that lives INSIDE a component — component owns it and can change it.

```
// Parent passes props
function Parent() {
const [name, setName] = useState('Faizan');
return ; // name is a prop to Child
}
// Child receives props — cannot modify them
function Child({ name }) {
return Hello {name};
}
```

Q: useContext — what is it?

➜ Shares data across many components without passing props through every level (prop drilling). Create a context, provide it at a high level, consume it anywhere below.

```
const AuthContext = React.createContext(null);
// Provider wraps your app
function App() {
const [user, setUser] = useState(null);
return
;
}
// Any nested component can access user
function Dashboard() {
const { user } = useContext(AuthContext);
return Welcome {user?.name};
}
```

Q: React component lifecycle (functional)

➜ Mount: component appears on screen → useEffect with [] runs. Update: state/props change → component re-renders → useEffect with [dep] runs. Unmount: component removed → cleanup function in useEffect runs.

```
useEffect(() => {
// MOUNT: runs once when component appears
const sub = subscribeToData();
return () => {
// UNMOUNT: cleanup when component disappears
sub.unsubscribe();
};
}, []);
```

N — Node.js

Q: What is Node.js? Why use it for backend?

➜ Node.js is a JavaScript runtime built on Chrome's V8 engine — lets you run JS outside the browser. Non-blocking I/O makes it fast for handling many concurrent connections. Same language as frontend = full-stack JS.

Q: What is the Node.js event loop?

➜ Node is single-threaded but handles concurrency via the event loop. When an async operation (file read, DB query) is started, Node doesn't wait — it registers a callback and moves on. When the operation completes, the callback runs.

```
// Non-blocking — Node continues to next line immediately
fs.readFile('file.txt', (err, data) => {
console.log(data); // runs when file is ready
});
console.log('This runs BEFORE file is read');
```

Q: require vs import — CommonJS vs ES Modules

➜ require() is CommonJS — original Node system. import/export is ES Modules — modern standard. Node supports both but you must be consistent within a project.

```
// CommonJS (older, .js files by default)
const express = require('express');
module.exports = router;
// ES Modules (add 'type':'module' in package.json or use .mjs)
import express from 'express';
export default router;
```

Q: Environment variables — .env file

➜ Store secrets (DB passwords, API keys, JWT secret) in a .env file. Never commit .env to GitHub. Access with process.env.VARIABLE_NAME.

```
// .env file
MONGO_URI=mongodb://localhost:27017/mydb
JWT_SECRET=mysecretkey123
PORT=5000
// In your code
require('dotenv').config();
const uri = process.env.MONGO_URI;
```

PART 2 — PYTHON

Q: Python basics — key differences from C++/JS

➜ No semicolons. Indentation defines blocks (not {}). Dynamic typing — no need to declare variable type. Everything is an object.

```
# Variables — no type declaration
name = 'Faizan'
age = 22
skills = ['Python', 'JS', 'ML']
# Function
def greet(name):
return f'Hello {name}'
# Class
class Person:
def __init__(self, name):
self.name = name
def speak(self):
return f'I am {self.name}'
```

Q: Lists, Tuples, Dicts, Sets — when to use each

➜ List: ordered, mutable, allows duplicates. Tuple: ordered, immutable, faster. Dict: key-value pairs, fast lookup. Set: unordered, unique elements only.

```
my_list = [1, 2, 2, 3] # mutable, ordered
my_tuple = (1, 2, 3) # immutable, ordered
my_dict = {'name': 'Faizan'} # key-value, fast lookup
my_set = {1, 2, 3, 2} # unique only → {1, 2, 3}
# List comprehension — Pythonic way to build lists
squares = [x**2 for x in range(10)] # [0,1,4,9,16...]
evens = [x for x in range(20) if x % 2 == 0]
```

Q: Lambda, map, filter, reduce

➜ Lambda: anonymous one-line function. map: apply function to every element. filter: keep elements that pass a test. reduce: accumulate into single value.

```
# Lambda
square = lambda x: x**2
square(5) # 25
# map — transform each element
nums = [1, 2, 3, 4]
doubled = list(map(lambda x: x*2, nums)) # [2,4,6,8]
# filter — keep matching elements
evens = list(filter(lambda x: x%2==0, nums)) # [2,4]
# reduce — accumulate
from functools import reduce
total = reduce(lambda a, b: a+b, nums) # 10
```

Q: Exception handling in Python

➜ Use try/except/finally. Catch specific exceptions — don't catch everything blindly.

```
try:
result = 10 / 0
except ZeroDivisionError as e:
print(f'Error: {e}')
except ValueError:
print('Wrong value')
finally:
print('Always runs — cleanup here')
# Raise your own exception
def divide(a, b):
if b == 0:
raise ValueError('Cannot divide by zero')
return a / b
```

Q: File handling — read and write

➜ Use 'with' statement — automatically closes file even if error occurs.

```
# Read file
with open('data.txt', 'r') as f:
content = f.read() # entire file as string
lines = f.readlines() # list of lines
# Write file
with open('output.txt', 'w') as f:
f.write('Hello Faizan')
# Append to file
with open('log.txt', 'a') as f:
f.write('New log entry\n')
```

Python for APIs — FastAPI & Django

Q: FastAPI vs Django — when to use which?

➜ FastAPI: lightweight, async, automatic docs, best for APIs and ML model serving. Django: full-featured framework with ORM, admin panel, auth — best for full web applications.

```
# FastAPI — minimal API
from fastapi import FastAPI
app = FastAPI()
@app.get('/predict')
async def predict(text: str):
result = model.predict(text)
return {'prediction': result}
# Auto docs at /docs — Swagger UI built in
```

Q: Django REST Framework — basic view

➜ DRF adds serializers and class-based views on top of Django for building REST APIs quickly.

```
# serializers.py
class UserSerializer(serializers.ModelSerializer):
class Meta:
model = User
fields = ['id', 'name', 'email']
# views.py
class UserListView(generics.ListCreateAPIView):
queryset = User.objects.all()
serializer_class = UserSerializer
```

PART 3 — AI / ML FUNDAMENTALS

Category	Meaning	Examples	Output type
Supervised	Train on labeled data — input + correct answer	Classification, Regression	Predicted label or value
Unsupervised	Find patterns in unlabeled data	Clustering, Dimensionality reduction	Groups or structure
Reinforcement	Agent learns by reward/punishment from environment	Game AI, Robotics	Action policy
Self-supervised	Model generates its own labels from data	BERT pretraining, GPT	Learned representations

Q: What is a model? What is training?

➜ A model is a mathematical function that maps inputs to outputs. Training = feeding data through the model, calculating error (loss), adjusting model weights (via backpropagation + gradient descent) to reduce error. Repeat until loss is low.

```
Training loop (conceptual):
for each batch of data:
prediction = model(input)
loss = calculate_error(prediction, true_label)
gradients = compute_gradients(loss)
update model weights using gradients
```

Q: What is overfitting? How do you fix it?

➜ Model memorizes training data — performs great on training, terrible on new data. Fix: more training data, dropout, regularization (L1/L2), simpler model, early stopping, cross-validation.

```
Signs: train accuracy 99%, test accuracy 60%
Fixes:
Dropout: randomly disable neurons during training
L2 Regularization: penalize large weights in loss function
Early stopping: stop training when val loss stops improving
More data: hard to overfit with enough diverse data
```

Q: Key ML algorithms — what they do

➜ Know what each algorithm is best at. You don't need to memorize math — know when to use what.

Algorithm	Type	Best for	Your projects
Linear Regression	Supervised	Predict continuous value (price, score)	—
Logistic Regression	Supervised	Binary classification baseline	—
Decision Tree	Supervised	Interpretable rules, tabular data	—
Random Forest	Supervised	Tabular data, handles noise	—
XGBoost	Supervised	Best for structured/tabular data	HealthMate FYP
K-Means	Unsupervised	Customer segmentation, grouping	—
PCA	Unsupervised	Reduce dimensions, remove noise	—
BiLSTM	Deep Learning	Sequence data — NLP, time series	NMT project
ResNet-50	Deep Learning	Image classification, feature extract	Image Captioning
GPT-2	Generative AI	Text generation, code generation	Pseudo2Code
BERT	Transformer	Classification, NER, QA — understanding	NLP coursework

Q: What is gradient descent?

➜ Algorithm for minimizing loss. Calculate gradient (direction of steepest increase in loss). Move weights in OPPOSITE direction by learning rate. Repeat until minimum found.

```
weight = weight - learning_rate * gradient
SGD: update after each single sample — noisy but fast
Mini-batch: update after small batch — best of both worlds
Adam: adaptive learning rate per parameter — most popular today
```

Q: What is backpropagation?

➜ Algorithm to compute gradients in a neural network. Forward pass: calculate prediction and loss. Backward pass: propagate error backwards through layers using chain rule to get gradient for each weight.

Q: Evaluation metrics — which to use when?

➜ Accuracy: only when classes are balanced. Precision: when false positives are costly. Recall: when false negatives are costly. F1: when both matter and classes are imbalanced. RMSE/MAE: for regression. BLEU: for text generation quality.

```
Precision = TP / (TP + FP) # of predicted positives, how many correct
Recall = TP / (TP + FN) # of actual positives, how many we caught
F1 = 2 * (P * R) / (P + R) # harmonic mean
BLEU score: n-gram overlap between generated and reference text
Your NMT project: BLEU 0.45+ (strong for seq-to-seq)
```

NLP — Natural Language Processing

Concept	What it does	Your experience
Tokenization	Split text into tokens (words/subwords)	Pseudo2Code, NMT
Embeddings	Map words to dense vectors (meaning)	FAISS retrieval, NMT attention
Attention	Model learns what to focus on	Bahdanau in BiLSTM NMT
Transformer	Self-attention — backbone of modern NLP	BERT, GPT-2, T5 via HuggingFace
BERT	Encoder-only, bidirectional — understands context	NLP coursework
GPT-2	Decoder-only, generates text left-to-right	Fine-tuned for Pseudo2Code
T5 / BART	Encoder-decoder — translation, summarization	NLP coursework
RAG	Retrieve docs + pass to LLM as context	Clinical RAG, AI Knowledge OS
FAISS	Vector DB — similarity search over embeddings	Digital Genei internship
Sentence Transformers	Produces sentence-level embeddings	Clinical RAG system
Whisper	Speech-to-text model by OpenAI	Digital Genei internship
BLEU Score	Measures text generation quality	NMT: 0.45+, Captioning: 0.1648

Q: What is RAG? When use it over fine-tuning?

➜ RAG = Retrieval Augmented Generation. At inference time, retrieve relevant documents using vector similarity, pass them as context to LLM, generate grounded answer. Use RAG when knowledge changes frequently, when you need citations, or when fine-tuning is too expensive. Fine-tune when you need to change model behavior/style permanently.

```
RAG Pipeline:
1. User asks question
2. Embed question → vector
3. Search vector DB (FAISS) → top-K relevant docs
4. Pass docs + question as context to LLM
5. LLM generates answer grounded in retrieved docs
```

Q: What is fine-tuning? How did you do it?

➜ Take a pretrained model (GPT-2). Train it further on your specific task/data with a low learning rate. Adapts the model's weights to your domain without training from scratch.

```
Your Pseudo2Code project:
Base model: GPT-2 (pretrained on internet text)
Your data: 14,500 pseudo-code → C++ pairs (SPOC dataset)
Training: causal language modeling, [PSEUDO]/[CODE] tokens
Precision: FP16 mixed-precision to save memory
Result: val loss 0.65, perplexity ~1.9
Output: model generates valid C++ from plain English
```

Q: What is a Vector Database? What is FAISS?

➜ Vector DB stores embeddings (dense vectors representing meaning). FAISS (Facebook AI Similarity Search) indexes these vectors and finds the most similar ones to a query vector using cosine similarity or L2 distance. Used in RAG, semantic search, recommendation systems.

```
// How it works:
1. Convert documents to vectors using embedding model
2. Store vectors in FAISS index
3. At query time: embed query → search FAISS → return top-K similar docs
Cosine similarity: measures angle between vectors — 1 = identical, 0 = unrelated
```

PART 4 — DATABASES & SYSTEM CONCEPTS

Concept	One-liner	Use when
Normalization	Remove redundancy — split into related tables	Write-heavy, need integrity
Denormalization	Add redundancy — reduce JOINs for faster reads	Read-heavy, analytics
Indexing	B-Tree structure on column for fast lookup	WHERE/JOIN/ORDER BY columns
ACID	Atomicity, Consistency, Isolation, Durability	Any transaction system
Transaction	Group of SQL ops that all succeed or all fail	Bank transfers, orders
Foreign Key	Column linking to PK of another table	Enforce relationships
Junction Table	Resolves M:N relationship with two foreign keys	Students-Courses, Orders-Products

Q: ACID — explain each with example

➜ Atomicity: bank transfer debit+credit both happen or neither does. Consistency: balance never goes negative. Isolation: two users booking same seat — only one succeeds. Durability: confirmed order persists even after server crash.

Q: Design a schema for a basic e-commerce system

➜ Start with entities, then relationships, then resolve M:N.

```
Users(user_id PK, name, email, password_hash, created_at)
Products(product_id PK, name, price, stock, category)
Orders(order_id PK, user_id FK, total, status, created_at)
OrderItems(item_id PK, order_id FK, product_id FK, qty, price)
// Relationships:
// User 1:M Orders | Order 1:M OrderItems
// Product M:N Order → resolved via OrderItems junction table
```

Q: Security basics every developer must know

➜ Never store plain text passwords — use bcrypt. Never put secrets in code — use .env. Validate all user inputs — prevent SQL injection. Use HTTPS. Set JWT expiry. Rate limit your APIs.

```
// Hash password before storing
const hashed = await bcrypt.hash(password, 10);
// Validate input — never trust user data
if (!email.includes('@')) return res.status(400).json({error});
// Rate limiting
const limiter = rateLimit({ windowMs: 15*60*1000, max: 100 });
app.use(limiter);
```

PART 5 — QUICK FIRE REFERENCE

Question	Answer
MongoDB vs MySQL	MongoDB = flexible docs, NoSQL. MySQL = structured tables, SQL, ACID.
React props vs state	Props = from parent, read-only. State = internal, component owns it.
JWT vs Session auth	JWT = stateless, token in header. Session = stateful, stored server-side.
REST vs GraphQL	REST = fixed endpoints. GraphQL = one endpoint, client specifies what it needs.
Node.js non-blocking	Async I/O via event loop — doesn't wait, registers callback, moves on.
Overfitting fix	More data, dropout, regularization, early stopping, simpler model.
BLEU score	Measures n-gram overlap between generated and reference text. Higher = better.
RAG vs fine-tuning	RAG = retrieve context at runtime. Fine-tune = bake knowledge into weights.
Precision vs Recall	Precision = quality of positives found. Recall = quantity of positives caught.
FAISS	Vector similarity search library. Finds nearest embeddings in O(log n).
Attention mechanism	Weights showing how much each input token matters for each output token.
Transformer encoder vs decoder	Encoder = understand (BERT). Decoder = generate (GPT). Both = translate (T5).
useEffect dependency array	[] = run once. [var] = run when var changes. nothing = run every render.
middleware in Express	Function between request and response. next() passes to next handler.
HTTP 401 vs 403	401 = not authenticated (no login). 403 = authenticated but no permission.

Connect every answer back to a project you built. That's what separates you from every other candidate.